Project Code

E-Commerce Sales Analysis - Full Python & SQL Code

Dataset: ecommerce_sales_messy.csv (3,035 rows, 14 columns). Copy-paste ready code, organized in the same order as the analysis: load → clean → transform → EDA, in both Python and SQL.

Part A: Python (pandas)

A1. Load the data

```
import pandas as pd
import numpy as np

df = pd.read_csv("ecommerce_sales_messy.csv")
print(df.shape)
print(df.dtypes)
print(df.head())
```

A2. Understand the data

```
print("=== MISSING VALUES ===")
print(df.isnull().sum())

print("=== DUPLICATES ===")
print("exact duplicate rows:", df.duplicated().sum())
print("duplicate Order_ID:", df.Order_ID.duplicated().sum())

print("=== TEXT INCONSISTENCY ===")
print("Region unique:", sorted(df.Region.dropna().unique()))
print("Payment_Method unique:", sorted(df.Payment_Method.dropna().unique()))

print("=== INVALID / OUTLIER VALUES ===")
print("negative qty:", (df.Quantity < 0).sum())
print("zero total amount:", (df.Total_Amount == 0).sum())
print("qty > 50:", (df.Quantity > 50).sum())

print("=== DATE ISSUES ===")
dt = pd.to_datetime(df.Order_Date, errors="coerce")
print("unparseable dates:", dt.isnull().sum())
```

A3. Clean the data (documented decisions)

```
start = len(df)

# 1. Drop exact duplicate rows
df = df.drop_duplicates()
print(f"Dropped {start - len(df)} exact duplicate rows")

# 2. Duplicate Order_ID -> keep first occurrence
before = len(df)
df = df.drop_duplicates(subset="Order_ID", keep="first")
print(f"Dropped {before - len(df)} duplicate Order_IDs")

# 3. Standardize text columns
df["Region"] = df["Region"].str.strip().str.title()
df["Payment_Method"] = df["Payment_Method"].str.strip().str.title()

# 4. Parse dates, drop rows with invalid dates
df["Order_Date"] = pd.to_datetime(df["Order_Date"], errors="coerce")
bad_dates = df["Order_Date"].isnull().sum()
df = df.dropna(subset=["Order_Date"])
print(f"Dropped {bad_dates} rows with invalid dates")

# 5. Negative quantity -> take absolute value (sign error)
```

```
df["Quantity"] = df["Quantity"].abs()

# 6. Zero Total_Amount -> can't be trusted, drop
df = df[df.Total_Amount != 0]

# 7. Outliers -> flag, don't drop (real customers, not noise)
q1, q3 = df.Quantity.quantile([.25, .75])
iqr = q3 - q1
upper = q3 + 1.5 * iqr
df["Qty_Outlier_Flag"] = df.Quantity > upper

# 8. Missing Unit_Price -> fill with category median
df["Unit_Price"] = df.groupby("Category")["Unit_Price"].transform(lambda x: x.fillna(x.median()))

# 9. Missing categorical fields -> fill with "Unknown"
for c in ["Region", "Payment_Method", "Delivery_Status"]:
    df[c] = df[c].fillna("Unknown")

# Rating & Customer_Email: left as NaN (optional fields, not fabricated)

print(f"Final cleaned shape: {df.shape}")
df.to_csv("ecommerce_sales_clean.csv", index=False)
```

A4. Exploratory analysis & insights

```
df = pd.read_csv("ecommerce_sales_clean.csv", parse_dates=["Order_Date"])

# Total / average
print("Total revenue:", df.Total_Amount.sum())
print("Average order value:", df.Total_Amount.mean())
print("Median order value:", df.Total_Amount.median())

# Category-wise revenue
print(df.groupby("Category")["Total_Amount"].sum().sort_values(ascending=False))

# Region-wise revenue
print(df.groupby("Region")["Total_Amount"].sum().sort_values(ascending=False))

# Monthly trend
monthly = df.groupby(df.Order_Date.dt.to_period("M"))["Total_Amount"].sum()
print(monthly)

# Payment method distribution
print(df.Payment_Method.value_counts())

# Delivery status & cancel/return rate
print(df.Delivery_Status.value_counts())
cancel_pct = df.Delivery_Status.isin(["Cancelled", "Returned"]).mean() * 100
print("Cancel/Return %:", round(cancel_pct, 1))

# Top 5 products by revenue
print(df.groupby("Product_Name")["Total_Amount"].sum().nlargest(5))

# Outlier impact check (why we flagged instead of dropping)
outlier_rev = df[df.Qty_Outlier_Flag].Total_Amount.sum()
print("Outlier revenue %:", round(outlier_rev / df.Total_Amount.sum() * 100, 1))
```

A5. Visualization

```
import matplotlib.pyplot as plt
import seaborn as sns

# Category revenue (bar)
cat_rev = df.groupby("Category")["Total_Amount"].sum().sort_values()
cat_rev.plot(kind="barh", figsize=(8, 5), title="Revenue by Category")
plt.show()

# Monthly trend (line)
```

```
monthly.plot(kind="line", marker="o", figsize=(10, 4), title="Monthly Revenue Trend")
plt.show()

# Order value distribution (seaborn)
sns.histplot(df["Total_Amount"], bins=20, kde=True)
plt.title("Order Value Distribution")
plt.show()
```

Part B: PostgreSQL

B1. Create table & import

```
CREATE TABLE ecommerce_sales (
    order_id          INT,
    customer_name     VARCHAR(100),
    customer_email    VARCHAR(150),
    category          VARCHAR(50),
    product_name      VARCHAR(100),
    quantity          INT,
    unit_price        NUMERIC(10,2),
    discount_percent  INT,
    total_amount      NUMERIC(10,2),
    order_date        TEXT,
    region            VARCHAR(50),
    payment_method    VARCHAR(50),
    delivery_status   VARCHAR(50),
    rating            NUMERIC(2,1)
);

\copy ecommerce_sales FROM 'ecommerce_sales_messy.csv' WITH (FORMAT csv, HEADER true);

SELECT COUNT(*) FROM ecommerce_sales;   -- should be 3035
```

B2. Data quality checks

```
-- Missing values
SELECT
    COUNT(*) - COUNT(customer_email)  AS missing_email,
    COUNT(*) - COUNT(unit_price)      AS missing_price,
    COUNT(*) - COUNT(region)          AS missing_region
FROM ecommerce_sales;

-- Exact duplicate rows
SELECT customer_name, customer_email, category, product_name, quantity,
       unit_price, total_amount, order_date, COUNT(*)
FROM ecommerce_sales
GROUP BY customer_name, customer_email, category, product_name, quantity,
         unit_price, total_amount, order_date
HAVING COUNT(*) > 1;

-- Duplicate Order_ID
SELECT order_id, COUNT(*)
FROM ecommerce_sales
GROUP BY order_id
HAVING COUNT(*) > 1;

-- Text inconsistency
SELECT DISTINCT region FROM ecommerce_sales ORDER BY region;
SELECT DISTINCT payment_method FROM ecommerce_sales ORDER BY payment_method;

-- Invalid values
SELECT * FROM ecommerce_sales WHERE quantity < 0;
SELECT * FROM ecommerce_sales WHERE total_amount = 0;
SELECT * FROM ecommerce_sales WHERE quantity > 50;
```

B3. Clean the data (create a clean table)

```
-- Fix invalid dates first
UPDATE ecommerce_sales
SET order_date = NULL
WHERE order_date !~ '^\d{4}-\d{2}-\d{2}$';

CREATE TABLE ecommerce_sales_clean AS
SELECT DISTINCT ON (order_id)
    order_id,
    customer_name,
    customer_email,
    category,
    product_name,
    ABS(quantity) AS quantity,
    unit_price,
    discount_percent,
    total_amount,
    order_date::date AS order_date,
    INITCAP(TRIM(region)) AS region,
    INITCAP(TRIM(payment_method)) AS payment_method,
    COALESCE(delivery_status, 'Unknown') AS delivery_status,
    rating
FROM ecommerce_sales
WHERE total_amount != 0
  AND order_date IS NOT NULL
ORDER BY order_id, order_date;
```

B4. Analysis queries

```
-- Total revenue & AOV
SELECT SUM(total_amount) AS revenue, AVG(total_amount) AS avg_order_value
FROM ecommerce_sales_clean;

-- Category-wise revenue
SELECT category, SUM(total_amount) AS revenue
FROM ecommerce_sales_clean
GROUP BY category
ORDER BY revenue DESC;

-- Monthly revenue trend
SELECT TO_CHAR(order_date, 'YYYY-MM') AS month, SUM(total_amount) AS revenue
FROM ecommerce_sales_clean
GROUP BY month
ORDER BY month;

-- Top 5 customers (window function)
SELECT customer_name, SUM(total_amount) AS total_spent,
       RANK() OVER (ORDER BY SUM(total_amount) DESC) AS rnk
FROM ecommerce_sales_clean
GROUP BY customer_name
LIMIT 5;

-- Orders above average (subquery)
SELECT * FROM ecommerce_sales_clean
WHERE total_amount > (SELECT AVG(total_amount) FROM ecommerce_sales_clean);

-- Cancel / return rate
SELECT ROUND(100.0 * SUM(CASE WHEN delivery_status IN ('Cancelled','Returned')
       THEN 1 ELSE 0 END) / COUNT(*), 1) AS cancel_return_pct
FROM ecommerce_sales_clean;

-- Top 5 products by revenue
SELECT product_name, SUM(total_amount) AS revenue
FROM ecommerce_sales_clean
GROUP BY product_name
ORDER BY revenue DESC
LIMIT 5;
```

B5. JOIN practice (sample customers table)

```
CREATE TABLE customers (
    customer_id INT PRIMARY KEY, customer_name VARCHAR(50), city VARCHAR(50)
);
INSERT INTO customers VALUES
(457,'Amit','Delhi'), (174,'Riya','Mumbai'), (364,'Karan','Bengaluru');

-- INNER JOIN: only matching rows
SELECT c.customer_name, SUM(o.total_amount) AS total_spent
FROM ecommerce_sales_clean o
JOIN customers c ON o.customer_name = c.customer_name
GROUP BY c.customer_name
ORDER BY total_spent DESC;

-- LEFT JOIN: all customers, even with zero orders
SELECT c.customer_name, COUNT(o.order_id) AS total_orders
FROM customers c
LEFT JOIN ecommerce_sales_clean o ON c.customer_name = o.customer_name
GROUP BY c.customer_name;
```